

TRABALHO EM GRUPO

# Sistema Integrado PetCare (SIP)

*Clínica Veterinária — Modelagem e Implementação em Java*

**Conceitos abordados: Herança · Encapsulamento · Composição · Delegação**

**Integrantes:**

- |          |                  |
|----------|------------------|
| 1. _____ | Matrícula: _____ |
| 2. _____ | Matrícula: _____ |
| 3. _____ | Matrícula: _____ |
| 4. _____ | Matrícula: _____ |
| 5. _____ | Matrícula: _____ |

Professor(a): \_\_\_\_\_

Data de entrega: \_\_\_\_ / \_\_\_\_ / \_\_\_\_

## 1. Contexto do Sistema

A Clínica Veterinária **PetCare** é referência no atendimento a animais domésticos na cidade de TechVille. Com o crescimento acelerado do número de clientes, a clínica vem enfrentando sérios problemas na organização interna, pois todos os processos ainda são realizados manualmente — uso de planilhas e anotações em papel —, ocasionando:

- Atrasos no agendamento de consultas e exames
- Erros no controle de estoque de medicamentos
- Falhas na comunicação com os tutores sobre vacinas, consultas e cirurgias
- Dificuldade na gestão financeira e na emissão de relatórios gerenciais

Para resolver esses desafios, a direção da clínica decidiu contratar uma equipe de desenvolvimento para criar o **Sistema Integrado PetCare (SIP)**, uma plataforma web unificada que abranja todos os processos essenciais da clínica.

## 2. Objetivo do Trabalho

O grupo deverá modelar e implementar, em linguagem Java, um sistema orientado a objetos que represente os principais módulos do SIP. Para isso, é necessário aplicar corretamente os quatro pilares de POO estudados:

|                       |                                                                                                                                                        |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Herança</b>        | Reutilizar comportamentos comuns entre classes por meio de extends, sobrescrevendo métodos com <code>@Override</code> quando necessário.               |
| <b>Encapsulamento</b> | Proteger os atributos com modificador <code>private</code> , expondo apenas getters e setters com validações de negócio.                               |
| <b>Composição</b>     | Representar relações "tem-um" declarando objetos como atributos de outras classes (ex: <code>Animal</code> possui <code>HistoricoClinico</code> ).     |
| <b>Delegação</b>      | Repassar responsabilidades a objetos colaboradores: a classe <code>Agendamento</code> não sabe enviar mensagens — delega ao <code>Notificador</code> . |

### 3. Diagrama de Classes UML

Os diagramas a seguir apresentam as classes do sistema com todos os atributos e métodos necessários. O grupo deve interpretá-los e implementar o código Java correspondente, respeitando os relacionamentos indicados.

#### 3.1 Hierarquia de Usuários (Herança + Encapsulamento)

A classe Usuario é a superclasse de todos os perfis de acesso do sistema. Cada subclasse herda os atributos e métodos comuns e acrescenta seus próprios comportamentos específicos.

| «superclasse»<br>Usuario                     |
|----------------------------------------------|
| - id : int                                   |
| - nome : String                              |
| - email : String                             |
| - senha : String                             |
| - perfil : String<br>{ADMIN VET RECEP TUTOR} |
| - ativo : boolean                            |
| - dataCadastro : String                      |
|                                              |
| + Usuario(id, nome, email, senha, perfil)    |
| + login(email, senha) : boolean              |
| + logout() : void                            |
| + getNome() : String                         |
| + getEmail() : String                        |
| + getPerfil() : String                       |
| + setNome(nome) : void {valida nulo}         |
| + setEmail(email) : void {valida @}          |
| + setSenha(senha) : void                     |
| + isAtivo() : boolean                        |
| + desativar() : void                         |
| + exibir() : void                            |

▼ extends ▼

| «extends Usuario»<br>Administrador     |
|----------------------------------------|
| - nivelAcesso : int                    |
| - departamento : String                |
|                                        |
| + Administrador(id,nome,email,senha)   |
| + excluirRegistro(id, tipo) : boolean  |
| + ajustarEstoque(item, qtd) : void     |
| + gerarRelatorio(mes, ano) : Relatorio |
| + gerenciarUsuario(u) : void           |

| «extends Usuario»<br>Veterinario     |
|--------------------------------------|
| - crmv : String                      |
| - especialidade : String             |
| - disponivel : boolean               |
|                                      |
| + Veterinario(id,nome,email,crm,esp) |
| + getCrmv() : String                 |
| + getEspecialidade() : String        |
| + isDisponivel() : boolean           |

|                               |
|-------------------------------|
| + visualizarLogs() : List     |
| + exhibir() : void {override} |

|                                    |
|------------------------------------|
| + setDisponivel(v) : void          |
| + registrarLaudo(id, texto) : void |
| + emitirPrescricao(id, rx) : void  |
| + consultarAgenda() : List         |
| + exhibir() : void {override}      |

| «extends Usuario»<br>Recepcionista   |
|--------------------------------------|
| - ramal : String                     |
| - turno : String                     |
|                                      |
| + Recepcionista(id,nome,email,ramal) |
| + cadastrarTutor(t) : void           |
| + cadastrarAnimal(a) : void          |
| + agendarConsulta(ag) : boolean      |
| + cancelarConsulta(id) : void        |
| + buscarTutor(termo) : Tutor         |
| + buscarAnimal(termo) : Animal       |
| + exhibir() : void {override}        |

| «extends Usuario»<br>Tutor      |
|---------------------------------|
| - cpf : String                  |
| - telefone : String             |
| - endereco : String             |
| - animais : List<Animal>        |
|                                 |
| + Tutor(id,nome,email,cpf,tel)  |
| + getCpf() : String             |
| + getTelefone() : String        |
| + getAnimais() : List<Animal>   |
| + adicionarAnimal(a) : void     |
| + verHistoricoAnimal(id) : void |
| + verFaturas() : List<Fatura>   |
| + buscarAnimal(nome) : Animal   |
| + exhibir() : void {override}   |

### 3.2 Módulo Clínico (Composição + Herança)

O módulo clínico representa o coração do sistema. A classe `Animal` possui (composição) um `HistoricoClinico`, que por sua vez agrega diferentes tipos de `RegistroClinico` — cada um implementado como uma subclasse especializada.

| «composição»<br>Animal               |
|--------------------------------------|
| - id : int                           |
| - nome : String                      |
| - especie : String                   |
| - raca : String                      |
| - dataNascimento : String            |
| - peso : double                      |
| - tutor : Tutor                      |
| - historico : HistoricoClinico       |
|                                      |
| + Animal(id,nome,especie,raca,tutor) |
| + getId() : int                      |
| + getNome() : String                 |
| + getTutor() : Tutor                 |
| + getHistorico() : HistoricoClinico  |
| + setPeso(peso) : void {> 0}         |
| + getPeso() : double                 |
| + calcularIdade() : int              |
| + exibir() : void                    |

| «composição»<br>HistoricoClinico |
|----------------------------------|
| - idAnimal : int                 |
| - consultas : List<Consulta>     |
| - vacinas : List<Vacina>         |
| - cirurgias : List<Cirurgia>     |
| - exames : List<Exame>           |
| - tratamentos : List<Tratamento> |
| - finalizado : boolean           |
|                                  |
| + HistoricoClinico(idAnimal)     |
| + adicionarConsulta(c) : void    |
| + adicionarVacina(v) : void      |
| + adicionarCirurgia(c) : void    |
| + adicionarExame(e) : void       |
| + adicionarTratamento(t) : void  |
| + finalizar() : void {RN07}      |
| + isFinalizado() : boolean       |
| + getConsultas() : List          |
| + getVacinas() : List            |
| + exibir() : void                |

▼ extends ▼ (todos herdam de RegistroClinico)

| «superclasse clínica»<br>RegistroClinico               |
|--------------------------------------------------------|
| - id : int                                             |
| - data : String                                        |
| - descricao : String                                   |
| - veterinario : Veterinario                            |
| - laudoAnexo : String                                  |
| - finalizado : boolean                                 |
|                                                        |
| + RegistroClinico(id, data, desc, vet)                 |
| + getId() : int                                        |
| + getData() : String                                   |
| + getVeterinario() : Veterinario                       |
| + finalizar() : void {bloqueia edição após finalizado} |
| + isFinalizado() : boolean                             |
| + anexarLaudo(path) : void                             |
| + getLaudo() : String                                  |

|                   |
|-------------------|
| + exibir() : void |
|-------------------|

▼ extends ▼

| «extends RegistroClinico»<br>Consulta |
|---------------------------------------|
| - motivo : String                     |
| - prescricao : String                 |
| - dataRetorno : String                |
|                                       |
| + Consulta(id,data,desc,vet,motivo)   |
| + getMotivo() : String                |
| + setPrescricao(rx) : void            |
| + getPrescricao() : String            |
| + setDataRetorno(dt) : void           |
| + getDataRetorno() : String           |
| + exibir() : void {override}          |

| «extends RegistroClinico»<br>Cirurgia |
|---------------------------------------|
| - duracao : int {min 120min}          |
| - sala : String                       |
| - equipe : List<String>               |
| - anestesia : String                  |
| - medicamentos : List<String>         |
|                                       |
| + Cirurgia(id,data,desc,vet,sala)     |
| + getSala() : String                  |
| + getDuracao() : int                  |
| + setEquipe(e) : void                 |
| + adicionarMedicamento(m) : void      |
| + validarRecursos() : boolean         |
| + exibir() : void {override}          |

| «extends RegistroClinico»<br>Exame |
|------------------------------------|
| - tipo : String                    |
| - resultado : String               |
| - imagemAnexo : String             |
| - laboratorio : String             |
|                                    |
| + Exame(id,data,desc,vet,tipo)     |
| + getTipo() : String               |
| + setResultado(r) : void           |
| + getResultado() : String          |
| + anexarImagem(path) : void        |
| + getImagem() : String             |
| + exibir() : void {override}       |

| «extends RegistroClinico»<br>Vacina   |
|---------------------------------------|
| - nomeVacina : String                 |
| - dataAplicacao : String              |
| - dataReforco : String                |
| - lote : String                       |
| - fabricante : String                 |
|                                       |
| + Vacina(id,data,desc,vet,nomeVacina) |
| + getNomeVacina() : String            |
| + getDataReforco() : String           |
| + setDataReforco(dt) : void           |
| + precisaReforco() : boolean          |
| + getLote() : String                  |
| + exibir() : void {override}          |

### 3.3 Módulo de Agendamento (Composição + Delegação)

A classe Agendamento possui (composição) referências a Animal, Veterinario e Notificador. Ao confirmar, cancelar ou reagendar, ela delega ao Notificador o envio das comunicações — sem saber como fazê-lo.

| «delegação»<br>Notificador          |
|-------------------------------------|
| - canal : String {EMAIL SMS APP}    |
| - destinatario : String             |
| - ativo : boolean                   |
|                                     |
| + Notificador(canal, destinatario)  |
| + getCanal() : String               |
| + setCanal(canal) : void            |
| + enviarConfirmacao(ag) : void      |
| + enviarCancelamento(ag) : void     |
| + enviarReagendamento(ag) : void    |
| + enviarLembreteVacina(a,dt) : void |
| + enviarAlertaEstoque(item) : void  |
| + enviarFatura(f) : void            |
| + enviarAlerta(dest, msg) : void    |
| + enviarCampanha(lista, msg) : void |

| «composição + delegação»<br>Agendamento           |
|---------------------------------------------------|
| - id : int                                        |
| - dataHora : String                               |
| - tipo : String {CONSULTA EXAME CIRURGIA}         |
| - status : String {AGENDADO CANCELADO CONCLUIDO}  |
| - animal : Animal                                 |
| - veterinario : Veterinario                       |
| - sala : String                                   |
| - notificador : Notificador                       |
| - historico : List<String>                        |
|                                                   |
| + Agendamento(id,dt,tipo,animal,vet,notif)        |
| + agendar() : boolean                             |
| + cancelar(motivo) : void                         |
| + reagendar(novaData) : void                      |
| + validarHorario() : boolean {08h-18h}            |
| + validarVeterinario() : boolean                  |
| + reservarRecursos() : boolean                    |
| + getStatus() : String                            |
| + getHistorico() : List                           |
| + notificarTutor() : void {delega ao Notificador} |
| + exibir() : void                                 |

### 3.4 Módulo de Estoque (Composição + Delegação)

| «encapsulamento»<br>ItemEstoque |
|---------------------------------|
| - id : int                      |
| - nome : String                 |
| - quantidade : int              |
| - quantidadeMinima : int        |
| - lote : String                 |
| - validade : String             |
| - controlado : boolean          |
| - responsavelRetirada : String  |
| - categoria : String            |
|                                 |

| «composição + delegação»<br>Estoque       |
|-------------------------------------------|
| - itens : List<ItemEstoque>               |
| - notificador : Notificador               |
|                                           |
| + Estoque(notificador)                    |
| + adicionarItem(item) : void              |
| + registrarEntrada(id, qtd, resp) : void  |
| + registrarSaida(id, qtd, resp) : boolean |
| + reservarParaProced(ag) : boolean        |

|                                        |
|----------------------------------------|
| + ItemEstoque(id,nome,qtd,qtdMin,lote) |
| + darEntrada(qtd, resp) : void         |
| + darSaida(qtd, resp) : boolean        |
| + reservar(qtd) : boolean              |
| + getQuantidade() : int                |
| + isAbaixoMinimo() : boolean           |
| + isControlado() : boolean             |
| + isVencido() : boolean                |
| + getLote() : String                   |
| + exhibir() : void                     |

|                                                   |
|---------------------------------------------------|
| + verificarAlertas() : void {delegao Notificador} |
| + buscarItem(nome) : ItemEstoque                  |
| + getItens() : List                               |
| + rastrearControlados() : List                    |
| + alertarVencimentos() : void                     |
| + gerarRelatorioRastreab() : void                 |
| + exhibir() : void                                |

### 3.5 Módulo Financeiro e Auditoria

| «composição + delegação»<br>Fatura           |
|----------------------------------------------|
| - id : int                                   |
| - tutor : Tutor                              |
| - valor : double                             |
| - dataEmissao : String                       |
| - dataVencimento : String                    |
| - status : String<br>{PENDENTE PAGO VENCIDO} |
| - descricaoServico : String                  |
| - notificador : Notificador                  |
|                                              |
| + Fatura(id,tutor,valor,desc,notif)          |
| + emitir() : void                            |
| + gerarBoleto() : String                     |
| + gerarLinkPagamento() : String              |
| + registrarPagamento() : void                |
| + confirmarPagOnline() : boolean             |
| + isPendente() : boolean                     |
| + getValor() : double                        |
| + getStatus() : String                       |
| + exibir() : void {delega ao Notificador}    |

| «encapsulamento»<br>Relatorio       |
|-------------------------------------|
| - id : int                          |
| - mes : int                         |
| - ano : int                         |
| - totalAtendimentos : int           |
| - faturamentoTotal : double         |
| - totalDespesas : double            |
| - taxaRetorno : double              |
| - tempMedioAtend : double           |
| - procedMaisRealizado : String      |
|                                     |
| + Relatorio(mes, ano)               |
| + gerar() : void                    |
| + getEstatisticas() : String        |
| + calcularFaturamento() : double    |
| + calcularTaxaRetorno() : double    |
| + calcularTempMedio() : double      |
| + getProcedMaisRealizado() : String |
| + exportar() : void                 |
| + exibir() : void                   |

| «composição»<br>LogAuditoria            |
|-----------------------------------------|
| - id : int                              |
| - dataHora : String                     |
| - usuario : Usuario                     |
| - acao : String                         |
| - entidadeAfetada : String              |
| - idEntidade : int                      |
| - ipOrigem : String                     |
|                                         |
| + LogAuditoria(usuario, acao, entidade) |
| + registrar() : void                    |
| + getAcao() : String                    |
| + getUsuario() : Usuario                |
| + getDataHora() : String                |
| + buscarPorUsuario(u) : List            |
| + buscarPorEntidade(e) : List           |
| + exibir() : void                       |

## 4. Tabela de Relacionamentos

A tabela abaixo sintetiza todos os relacionamentos entre as classes do sistema. O grupo deve garantir que cada um deles esteja corretamente implementado no código Java.

| Classe                  | Relacionamento | Com                         | Justificativa                                             |
|-------------------------|----------------|-----------------------------|-----------------------------------------------------------|
| <b>Usuario</b>          | Herança        | Admin / Vet / Recep / Tutor | Superclasse com login, perfil e encapsulamento (RF24)     |
| <b>Administrador</b>    | Herança        | Usuario                     | Acrescenta excluirRegistro(), ajustarEstoque() (RN10)     |
| <b>Veterinario</b>      | Herança        | Usuario                     | Acrescenta registrarLaudo(), emitirPrescricao() (RF12)    |
| <b>Recepcionista</b>    | Herança        | Usuario                     | Acrescenta cadastrarTutor(), agendarConsulta() (RF06)     |
| <b>Tutor</b>            | Herança        | Usuario                     | Acrescenta getAnimais(), verFaturas() (RF25)              |
| <b>Animal</b>           | Composição     | Tutor                       | Animal tem-um Tutor principal (RN01)                      |
| <b>Animal</b>           | Composição     | HistoricoClinico            | Animal tem-um HistoricoClinico único (RF03)               |
| <b>HistoricoClinico</b> | Composição     | RegistroClinico             | Agrega listas de Consulta, Cirurgia, Exame, Vacina (RF03) |
| <b>Consulta</b>         | Herança        | RegistroClinico             | Especializa com motivo, prescrição, retorno (RF12)        |
| <b>Cirurgia</b>         | Herança        | RegistroClinico             | Especializa com sala, equipe, duração mínima (RN02)       |
| <b>Exame</b>            | Herança        | RegistroClinico             | Especializa com resultado e imagem anexa (RF04)           |
| <b>Vacina</b>           | Herança        | RegistroClinico             | Especializa com lote, reforço, fabricante (RF27)          |
| <b>Agendamento</b>      | Composição     | Animal + Vet                | Associa procedimento a animal e veterinário (RF06)        |
| <b>Agendamento</b>      | Delegação      | Notificador                 | Repassa envio de confirmação/cancelamento (RF10)          |
| <b>Estoque</b>          | Composição     | ItemEstoque                 | Gerencia lista de itens com reserva automática (RF17)     |
| <b>Estoque</b>          | Delegação      | Notificador                 | Repassa alertas de estoque mínimo (RF16)                  |
| <b>Fatura</b>           | Composição     | Tutor                       | Fatura pertence a um Tutor (RF20)                         |

|                     |            |             |                                                    |
|---------------------|------------|-------------|----------------------------------------------------|
| <b>Fatura</b>       | Delegação  | Notificador | Repassa envio de boleto e link de pagamento (RF20) |
| <b>LogAuditoria</b> | Composição | Usuario     | Registra ação com o usuário responsável (RN15)     |

## 5. Requisitos de Implementação

A implementação em Java deve atender obrigatoriamente a todos os itens abaixo.

### 5.1 Estrutura mínima obrigatória

1. Implementar todas as 14 classes do diagrama UML com atributos e métodos descritos.
2. Todos os atributos devem ser declarados com modificador `private`.
3. Getters e setters devem conter validações de negócio adequadas (ex: peso > 0, email com @).
4. Construtores parametrizados conforme o diagrama.
5. Métodos `@Override` nas subclasses com comportamento específico.
6. A classe `Main` deve instanciar objetos de todas as classes e testar os métodos principais.

### 5.2 Regras de negócio a implementar

| Regras obrigatórias no código                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------|
| • RN01 — Cada Animal pertence a apenas um Tutor principal (atributo <code>tutor</code> : Tutor).                                           |
| • RN02 — Cirurgia deve ter duração mínima de 120 minutos (validar no construtor ou setter).                                                |
| • RN03 — <code>Agendamento.validarVeterinario()</code> deve verificar disponibilidade ( <code>disponivel == true</code> ).                 |
| • RN04 — <code>Agendamento.validarHorario()</code> deve aceitar apenas horários entre 08h e 18h.                                           |
| • RN07 — <code>HistoricoClinico.finalizar()</code> deve bloquear adição de novos registros após finalização.                               |
| • RN08 — <code>ItemEstoque.isAbaixoMinimo()</code> deve retornar <code>true</code> quando <code>quantidade &lt; quantidadeMinima</code> .  |
| • RN09 — <code>ItemEstoque</code> controlado deve exigir lote, validade e responsável não nulos.                                           |
| • RN11 — <code>Vacina.precisaReforco()</code> deve retornar <code>true</code> quando a <code>dataReforco</code> for anterior à data atual. |
| • RN12 — <code>Fatura.confirmarPagOnline()</code> simula retorno de API: sempre retorna <code>true</code> nesta versão.                    |
| • RN15 — <code>LogAuditoria.registrar()</code> deve gravar data/hora, usuário e ação.                                                      |

### 5.3 Exemplo de saída esperada na classe Main

```

=== SISTEMA PETCARE ===
[USUARIO] Login: ana@petcare.com — Perfil: VET
[ANIMAL] Rex | Labrador | Tutor: João Silva
[HISTORICO] Consultas: 2 | Vacinas: 1 | Cirurgias: 0
[AGENDAMENTO] Tipo: CONSULTA | Status: AGENDADO
[NOTIFICADOR] EMAIL → joao@email.com: Consulta confirmada!
[ESTOQUE] Dipirona | Qtd: 50 | Mínimo: 10 → OK
[FATURA] Valor: R$150,00 | Status: PENDENTE
[LOG] 2025-03-10 09:15 | Usuario: admin | Ação: AJUSTE_ESTOQUE

```

## 6. Entregáveis

- Código-fonte Java (.java) de todas as classes, organizado em pacotes.
- Relatório de 2 páginas explicando onde e como cada pilar de POO foi aplicado.

## 7. Critérios de Avaliação

- Código-fonte Java (.java) de todas as classes, organizado em pacotes. (2,0)
- Relatório de 2 páginas explicando onde e como cada pilar de POO foi aplicado. (1,0)

## 8. Dicas Importantes

### Antes de começar a codificar

- Leia o diagrama UML completo antes de escrever uma linha de código.
- Implemente na ordem de dependência: primeiro as classes sem dependências (Notificador, ItemEstoque), depois as que dependem delas.
- Ordem sugerida: Notificador → ItemEstoque → Usuario → subclasses → RegistroClinico → subclasses → HistoricoClinico → Animal → Estoque → Agendamento → Fatura → Relatorio → LogAuditoria → Main.
- Teste cada classe isoladamente antes de integrá-la com as demais.

### Erros comuns a evitar

- Declarar atributos como public — isso viola o encapsulamento.
- Esquecer o super() no construtor das subclasses.
- Criar cópias dos mesmos métodos nas subclasses em vez de usar herança.
- Fazer a classe Agendamento enviar e-mails diretamente em vez de delegar ao Notificador.
- Não validar os setters (peso negativo, e-mail sem @, etc.).